

EXPERIMENT 1

Write a OpenMP program to sort an array on n elements using both sequential and parallel merge sort (using Section). Record the difference in execution time.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

// Merge Function
void merge(int arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    // Copy Left subarray
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    // Copy right subarray
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // Merge the temporary arrays
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    // Copy remaining elements of L[]
    while (i < n1)
        arr[k++] = L[i++];

    // Copy remaining elements of R[]
    while (j < n2)
        arr[k++] = R[j++];
}

// Sequential Merge Sort
```

```

void mergeSortSequential(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        mergeSortSequential(arr, left, mid);
        mergeSortSequential(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

```

// Parallel Merge Sort

```

void mergeSortParallel(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            mergeSortParallel(arr, left, mid);

            #pragma omp section
            mergeSortParallel(arr, mid + 1, right);
        }

        merge(arr, left, mid, right);
    }
}

```

```

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr1 = (int *)malloc(n * sizeof(int));
    int *arr2 = (int *)malloc(n * sizeof(int));

    printf("Enter %d elements: ", n);

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr1[i]);
    }
}

```

```

        arr2[i] = arr1[i];    // Copy for parallel sorting
    }

    double start, end;

    // Sequential Merge Sort
    start = omp_get_wtime();
    mergeSortSequential(arr1, 0, n - 1);
    end = omp_get_wtime();

    printf("Sequential Merge Sort Time: %f seconds\n", end - start);

    // Parallel Merge Sort
    start = omp_get_wtime();
    mergeSortParallel(arr2, 0, n - 1);
    end = omp_get_wtime();

    printf("Parallel Merge Sort Time: %f seconds\n", end - start);

    printf("Sorted array: ");

    for (int i = 0; i < n; i++)
        printf("%d ", arr1[i]);

    printf("\n");

    free(arr1);
    free(arr2);

    return 0;
}

```

OUTPUT:

ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~\$ gcc -fopenmp mergesort.c

ubuntu@ubuntu-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~\$./a.out

Enter number of elements: 6

Enter 6 elements: 2 9 1 5 10 4

Sequential Merge Sort Time: 0.000001 seconds

Parallel Merge Sort Time: 0.000244 seconds

Sorted array: 1 2 4 5 9 10